

Introduction to BASH scripting

Objective: In this lab you will be introduced to the basics of bash scripting.

Learning goals: After this lab you will be able to:

1. understand basic concepts of a script
2. apply logic to resolve common tasks with minimal commands
3. create a basic to intermediate bash script
4. execute and run a basic to intermediate script

Complete the following tasks using the examples in each question. ****As with all labs make sure you take a snapshot before you start.***

Commands: **nano, vim, touch, sh, mkdir -p, ./ tar**

Prepwork:

1. Create a directory in your home folder called **scripts**
2. Open a text editor. *gedit (graphical), nano (cli), or vim (cli) **vim is not installed by default*

Creating the script:

Script 1: **Hello World**. This script you will echo back **Hello World**.

In your text editor window: Type the **highlighted** lines: **Do not enter the italic lines, they are for your understanding of what the command is doing.*

1. **#!/bin/bash**
This line is called the "sh-bang" and is used to show it is a bash shell script.
2. **clear**
3. **echo "Hello World"**
This will print Hello World on the screen.
4. Save the file as **helloworld.sh** to the scripts folder and open the terminal (if you are not already in terminal).
5. Run the script using one of the options below.
6. Run the script again, but this time redirect the output to a file **hello.txt**
**use the redirect > to direct the output to the file.*

Running the script:

In the next steps you will run your script. There are two ways to do this. Use one of the following ways to test your script.

Option 1: Set the file as an executable by using **chmod** to change the execute bit to on.

1. **chmod +x helloworld.sh**
This sets the execute bit to on. Make sure you are in the scripts directory to run this correctly.
2. **./helloworld.sh**
This command runs the executable helloworld.sh script

Introduction to BASH scripting

Option 2: Run the file using **sh**. This does **NOT** require you to change permissions. The **.sh** extension was used to show the utility needed instead of using strictly permissions to tell if it is executable. If the script were a perl script, the **.pl** extension would have been used.

1. **sh helloworld.sh**

This is the command used to run the script.

Script 2: Current User. This script will check the current user. **Open a new editor page for the new script. Remember to do this for each script.*

1. **#!/bin/bash**
2. **clear**
3. **echo "The current user is: \$USER "**
4. Save the file as **user.sh** and use one of the previous options to run the script.
5. Run the script again, but this time redirect the output to a file called **user.txt**
sh user.sh > user.txt

Script 3: Date and Time. This script will output the date and time.

1. **#!/bin/bash**
2. **clear**
3. **echo "The current date and time is:" `date`**
Notice the ` is NOT ' there is a difference!
4. Save the file as **datetime.sh** and use one of the options to run the script.
5. Run the script again, but this time redirect the output to a file called **datetime.txt**

Script 4: Variable and Input. This script will set a variable to call and allow for user input.

1. **#!/bin/bash**
2. **clear**
3. **MY_NAME = "foo bar"**
4. **echo "Enter your real name here: "**
5. **read realname**
6. **echo "Hello my name is \$MY_NAME"**
7. **echo** **This just puts a space between lines to make it look pretty.*
8. **echo "Liar, my name is \$realname"**
9. **echo** **This just puts a space between lines to make it look pretty.*
10. **echo "Okay, goodbye \$realname"**
11. **echo "end of script"**
12. Save the file as **varin.sh** and use one of the options to run the script.
13. Screen shot or copy and paste the screen process for the script and save the screen shot or text file as **varin.png (jpg, bmp, tiff, etc) or varin.txt** **only save it as .txt if it is in a text file.*

Introduction to BASH scripting

Script 5: System variables. This script will output information from the system variables defaulted in the system. *****Note: DO NOT change the system variables. This can cause problems in your system.***

Example: Do NOT change USER to something else or it will mess with other functions and cause system kernel panics.

1. `#!/bin/bash`
2. `clear`
3. `echo "The system time is: "`time``
4. `echo`
5. `echo "The current logged in users are: "`who``
6. `echo`
7. `echo "The current date and time is: "`date``
8. `echo`
9. `echo "The user currently logged in is: $USER"`
10. `echo`
11. `echo "Goodbye"`
12. Save the file as **system.sh** and use one of the options to run the script.
13. Run the script again and redirect the output to a file called **system.txt**

Script 6: if else. This will show you how to write a simple if then else script. This script type is used to help qualify an either or type question.

1. `#!/bin/bash`
2. `clear`
3. `directory= "../scripts" *this sets the directory variable`
4. `# bash check if directory exists`
5. `if [ -d $directory ]; then`
 - a. `echo "Directory exists"`
6. `else`
 - a. `echo "Directory does not exist"`
7. `fi`

*****Note: when you start with if you have to end with fi Also make sure you tab in on the echo to make it appear like it does above. Make sure you are ONLY copying the highlighted lines, do NOT put the numbers or letters in the scripts!***

8. Save the file as **ifelse.sh** and use an option above to run the script.
9. Run the script again and redirect the output to a file called; **ifelse.txt**

Introduction to BASH scripting

Script 7: Nested if else. This will show you how to nest **if/else** statements for conditional scripts. This allows you to ask questions and qualifiers to accomplish the task.

```
1. #!/bin/bash
2. clear
3. # Declare variable choice and assign value 4
4. choice=4
5. # Print to screen (stdout)
6. echo " 1. Bash"
7. echo "2. Scripting"
8. echo "3. Tutorial"
9. echo -n "Please choose a word [1, 2, or 3]? "
10. # Loop while the variable choice is equal 4
11. # bash while loop
12. while [ $choice -eq 4 ]; do
13. # read user input
14. read choice
15. # bash nested if/else
16. if [ $choice -eq 1 ]; then
17.     a. echo "You may have chosen word: Bash"
18. else
19.     a. if [ $choice -eq 2 ]; then
20.         i. echo "You have chosen word: Scripting"
21.     else
22.         i. if [ $choice -eq 3 ]; then
23.             1. echo "You may have chosen word: Tutorial"
24.         else
25.             1. echo "Please make a choice between 1-3 !"
26.             2. echo "1. Bash"
27.             3. echo "2. Scripting"
28.             4. echo "3. Tutorial"
29.             5. echo -n "Please choose a word [1, 2, or 3]"
30.             6. choice =4
31.         fi
32.     fi
33. fi
34. done
```

20. Save the file as **ifelsenest.sh** and run the script with one of the options above.

21. Take a screen shot of the script run or copy and paste the script input/output. Save the screen shot or text copy as **ifelsenest.png** or **ifelsenest.txt**

Introduction to BASH scripting

Script 8: Making menus with the `case` statement. This simplifies the nesting of conditional if statements.

1. `#!/bin/bash`
2. `clear`
3. `# Display the information for the menu and how to select options.`
4. `echo "Welcome to the Cyber Café!"`
5. `echo "" #Creating a space between lines.`
6. `echo "Please check out our menu and make your selections."`
7. `echo ""`
8. `# Set an exit variable like x=0 (zero)`
9. `x=0`
10. `while [ $x = 0 ]; do`
 - a. `echo -n "Do you want a beverage? (y/n): "`
 - b. `read ans`
 - c. `case "$ans" in`
 - i. `y)` `#This will respond to a "y" $ans value and run the commands`
 - ii. `echo "Great! What would you like to drink?"`
 - iii. `echo "Choose one of the listed items below using (1, 2, 3, or 4)"`
 - iv. `echo "1 – Coffee, 2 – Cappuccino, 3 – Espresso, 4 – Latte"`
 - v. `echo ""`
 - vi. `echo -n "Enter your selection here: "` `#The -n will keep the input on the same line`
 - vii. `read choice`
 - viii. `y=0 #Setting another exit variable to escape the loop when ready.`
 - ix. `while [ $y = 0 ]; do`
 1. `case "$choice" in`
 - a. `1)`
 - b. `echo "Coffee"`
 - c. `echo ""`
 - d. `y=1 #This will make "y" != 0 and will exit the loop.`
 - e. `;; #This will end this part of the 1) statement and allow for 2)`
 - f. `2)`
 - g. `echo "Cappuccino"`
 - h. `echo ""`
 - i. `y=1`
 - j. `;;`
 - k. `3)`
 - l. `echo "Espresso"`
 - m. `echo ""`
 - n. `y=1`
 - o. `;;`
 - p. `4)`
 - q. `echo "Latte"`
 - r. `echo ""`
 - s. `y=1`
 - t. `;;`

Introduction to BASH scripting

```
u. *)      #This is used as a catch if anything other than 1-4 is entered.
v. echo "Please enter a selection: 1, 2, 3, or 4: "
w. read choice
x. clear
y. ;;

esac #Case spelled backwards to complete the statement
done
x=1 #This will exit the first while loop and allow for a no (n) $ans response
;; #This ends the y) condition and is necessary to move to n)
n)
echo "Okay, Thank you!"
echo ""
x=1
;;
*)
echo -n "Please choose y or n to continue: "
read ans
clear
;;

esac

done
```

Practical Exercises:

Complete the following steps by creating a script to complete the requested output.

Exercise 1: Write a script with the following... Set up the user inputs with requests. For example if you want someone to enter something: Please enter ...

- clear the screen
- enter a word
- repeat what the word is
- enter two words
- show what the input entered was
- ask how you feel about bash scripting
- show what the response was
- enter five favorite colors
- show the five favorite colors
- agree the five colors are cool and repeat the colors

Save the script as **exe1.sh** and run it using one of the options

Introduction to BASH scripting

Take a screen shot or copy and paste to a file named **exe1.png** (or other photo extension) or **exe1.txt**

Exercise 2: Write a script with the following... Set up the input and outputs like you were having a conversation.

- Clear the screen
- introduce yourself
- respond back cordially and repeat the name
- give your name
- ask what they would like to drink
- give a response
- repeat response
- ask what size 12oz 16oz 20oz or 24oz **Note: you may want to set variables or array for the sizes or load to \$REPLY*
- repeat drink and size
- ask if they want any of the following straw, cream, sugar, ice, whipped cream
- Ask if they would like anything else **Note: Use a nested approach.*
- If nothing else, call name and repeat order
- give thanks and tell them to have a nice morning, afternoon, evening, or day **Note: you may want to create a variable for this to call*

Save the script as **exe2.sh** and run the script using an option previously discussed.

Take screen shots or copy and paste the input/output from this script and save them as **exe2.png** or **exe2.txt**

Exercise 3: Write a script with the following... Create a menu for installing the following packages using the case statement.

- Apache2
- VSFTPD
- mysql-server
- python3
- acl
- all of the above

Give them the option to install individual packages or all of them. Keep it simple and just run one option for this, but know that you can make this so each install would break out into a new bash shell if required. For this exercise keep it simple!

Save the script as **exe3.sh** and run the script using an option previously discussed.

Take screen shots or copy and paste the input/output from this script and save them as **exe3.png** or **exe3.txt**

Introduction to BASH scripting

Compress the **scripts** directory and upload it to the 4.0 bash assignment. Verify all files are in the **scripts** directory. *Review each step. There are 7 scripts that have 2 parts to each totaling 14 files plus the 2 exercises totaling 4 files.

Total files = 18 files

Archive the **scripts** directory using the command: **You can use the gui archive as well.*

tar -cjvpf scripts.tar.bz2 scripts

Test your archive by using: **You can use the gui archive viewer as well.*

tar -tjvpf scripts.tar.bz2